SOFTWARE REQUIREMENTS SPECIFICATION

for

Smart Hostel Room Allocation System

Version 1.0

Prepared by:

1. Avleen Kaur (1024030403)
2. Rishab Kumar Jha (1024030416)
3. Daksh Kohli (1024030423)

Submitted to: Mr. Hardik

Thapar Institute of Engineering and Technology

Contents

1 Introduction **2**
- 1.1 Purpose . . . 2
- 1.2 Intended Audience and Reading Suggestions . . . 2
- 1.3 Project Scope . . . 2

2 Overall Description **5**
- 2.1 Product Perspective . . . 5
- 2.2 User Classes and Characteristics . . . 5
- 2.3 Product Functions . . . 5
- 2.4 Operating Environment . . . 5
- 2.5 Design and Implementation Constraints . . . 5
- 2.6 Assumptions and Dependencies . . . 6

3 System Features **7**
- 3.1 Description and Priority . . . 7
- 3.2 Functional Requirements . . . 7

4 External Interface Requirements **9**
- 4.1 User Interfaces . . . 9
- 4.2 Hardware Interfaces . . . 9
- 4.3 Software Interfaces . . . 9
- 4.4 Communications Interfaces . . . 9

5 Non-Functional Requirements **10**
- 5.1 Performance . . . 10
- 5.2 Security . . . 10
- 5.3 Reliability & Availability . . . 10
- 5.4 Maintainability . . . 10
- 5.5 Portability . . . 10
- 5.6 Usability . . . 10
- 5.7 Business Rules . . . 10

6 System Architecture (High-Level) **11**

7 Data Model (Entity Overview) **12**

8 Allocation Algorithm — Detailed Logic **13**

9 Appendix **14**
- 9.1 Tech Stack Summary . . . 14

1. Introduction

1.1 Purpose

Hostel administration at the institute already runs on FretBox, which holds student master data and handles fees, complaints and gate passes. What FretBox does not provide is student-driven room allocation: rooms are assigned centrally, friend groups coordinate informally over messaging apps, and students learn their allotment only at check-in. The Smart Hostel Room Allocation System closes that gap. It draws student and hostel records from FretBox rather than any hand-maintained sheet, and lets students form groups and self-select clusters inside an automated, rule-governed allocation engine.

1.2 Intended Audience and Reading Suggestions

This SRS is written for faculty evaluators, the student development team, and future maintainers. Sections 2–3 describe the system functionally; Sections 4–8 give the technical detail needed to build and maintain it.

1.3 Project Scope

The system allows:

- Students register against their FretBox record, then create or join a group using a group code.
- Group leaders browse live cluster availability and submit a ranked list of preferences.
- Rooms are allocated automatically to groups and single applicants by availability and preference order.
- Every group member must complete fee payment before the booking is confirmed.
- Admins can override allocations and monitor bookings and payments.

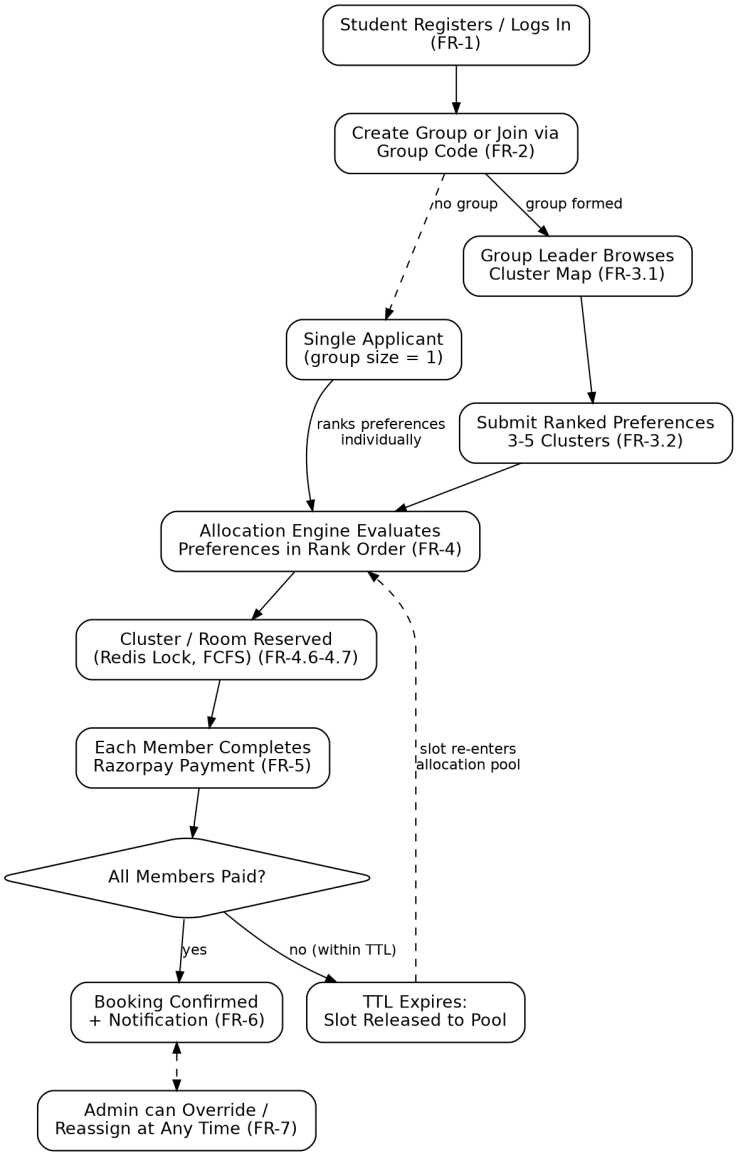


Figure 1.1: End-to-end student booking workflow, from registration to confirmed booking.

Definitions, Acronyms, and Abbreviations

Term	Definition
Cluster	Two rooms sharing a bathroom; total capacity 4 students
Room	A 2-seater unit; two rooms per cluster
Group	1–4 students who wish to be housed together, led by a Group Leader
FCFS	First-Come-First-Served — the tie-break rule for conflicting cluster preferences

References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- Razorpay API documentation; FretBox data-export documentation

2. Overall Description

2.1 Product Perspective

A containerized web application layered on the institute's existing FretBox deployment. A Next.js + Tailwind CSS frontend talks to a FastAPI REST backend, with PostgreSQL for persistence and Redis for booking-slot locking. Student and hostel master data is imported from FretBox; payments are processed by Razorpay. The full stack is orchestrated by Docker Compose.

2.2 User Classes and Characteristics

User Class	Description
Student	Creates or joins a group, ranks cluster preferences, pays own fee. A student with no group is treated as a group of one.
Admin / Warden	Manages the FretBox data sync, overrides allocations, resolves exceptions.

2.3 Product Functions

- Authentication with JWT sessions and student/admin roles; profiles seeded from FretBox.
- Group creation and joining by group code, managed by the group leader.
- Floor-wise visual cluster map with live occupancy, built from imported hostel data, with ranked preference submission (3–5 clusters) by the leader.
- Per-student fee payment via Razorpay, gating booking confirmation.
- Core allocation algorithm covering all group-size and occupancy combinations, made race-free by Redis distributed locks.
- Admin dashboard for allocation overrides and system-wide booking and payment monitoring.

2.4 Operating Environment

- Server: Dockerized Linux containers (backend, frontend, PostgreSQL, Redis).
- Client: any modern browser, desktop or mobile. A single `docker compose up` starts every service.

2.5 Design and Implementation Constraints

- Backend implemented in FastAPI (Python) with async endpoints.
- Frontend in Next.js (App Router) with Tailwind CSS, using card-based booking-platform visual conventions.
- All payments go through Razorpay's official SDK; no card data is handled directly.

- Occupancy state changes must be atomic — Redis distributed locks (SETNX / Redlock) guard the allocation critical section.
- Hostel, floor, cluster and room records are imported from FretBox; the system holds no hand-entered master data.

2.6 Assumptions and Dependencies

- Each hostel houses a single gender and academic year, so the allocation engine performs no gender or year checks.
- Razorpay test-mode credentials are used for the demonstration.
- Group size is capped at 4, matching cluster capacity, and FretBox is assumed to expose current student and hostel data.

3. System Features

3.1 Description and Priority

Features are listed in descending priority: (1) the core allocation algorithm, (2) Redis-locked booking with FCFS conflict resolution, (3) group management, (4) the cluster map and preference submission, (5) fee payment, (6) authentication and profile, and (7) the admin module. The allocation engine is the core deliverable; everything else exists to get a group safely through it.

3.2 Functional Requirements

FR-1: Authentication & Profile

- FR-1.1: Students register with their institute email and roll number; profile fields (branch, year, gender, eligibility) come from FretBox, and students can view their allocation and payment status at any time.
- FR-1.2: JWT-based login and logout, with a role claim distinguishing student from admin.

FR-2: Group Management

- FR-2.1: A student may create a group and become its Group Leader; the system generates a unique group code. A student with no group is treated internally as a group of size 1.
- FR-2.2: Other students join by entering the group code, up to a maximum of 4 members.
- FR-2.3: The leader may remove members or disband the group before booking confirmation.

FR-3: Cluster/Room Visual Map & Preferences

- FR-3.1: A floor-wise visual layout renders each cluster's two rooms with colour-coded live occupancy (Empty / Partially Filled / Full).
- FR-3.2: The Group Leader ranks 3 to 5 preferred clusters; submission is permitted only once all members have joined.

FR-4: Allocation Algorithm (Core)

- FR-4.1 to FR-4.3: Size-matching rules — a 4-member group requires both rooms of the cluster empty; a 3-member group requires the leader's room empty and the connected room either empty (leaving one seat for a future single) or holding exactly one occupant; a 2-member group requires only the leader's room to be empty.
- FR-4.4: A single applicant is placed in the first preferred cluster with exactly one free seat; if none qualifies, the system falls back to a system-wide search.
- FR-4.5: If no ranked preference fits, the system proposes the next feasible cluster or prompts the leader to re-rank.
- FR-4.6: All checks and writes for a cluster occur inside a Redis lock scoped to that cluster ID, ensuring atomicity under concurrent booking attempts.
- FR-4.7: Conflicts are resolved strictly by FCFS, timestamped at the moment a group becomes booking-ready (preferences submitted and all member fees paid).

FR-5: Fee Payment (Razorpay Integration)

- FR-5.1: Each member completes hostel fee payment independently via Razorpay; a per-group Payment Status view shows each member's paid/unpaid state.
- FR-5.2: A cluster moves from Reserved to Confirmed only when every member's payment status is paid; otherwise the reservation lock expires and the slot returns to the pool.

FR-6: Booking Confirmation

- FR-6.1: On successful allocation and full payment, a booking confirmation record is generated and all members are notified.
- FR-6.2: Booking states: Pending Preferences → Reserved (Awaiting Payment) → Confirmed → (optionally) Reassigned by Admin.

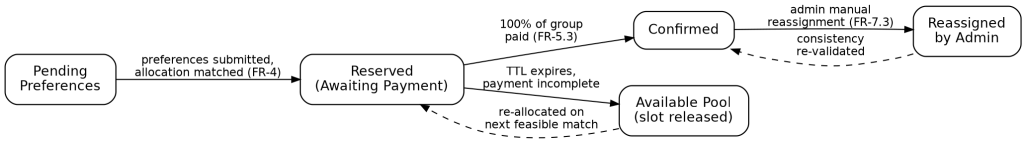


Figure 3.1: Booking status state diagram, including TTL-driven release back to the available pool.

FR-7: Admin Module

- FR-7.1: Admin can edit hostel, floor, cluster and room records and re-sync them from FretBox.
- FR-7.2: Admin can view all allocations, filtered by hostel, floor, cluster or payment status, and run payment reconciliation reports.
- FR-7.3: Admin can manually reassign or vacate a student or group, triggering allocation consistency checks.

4. External Interface Requirements

4.1 User Interfaces

- Landing / Auth: login and signup with an imagery-led hero section.
- Dashboard and Cluster Browser: group status and profile summary, plus a card-based floor map with live occupancy badges.
- Preference Ranking and Payment: drag-to-rank cluster selection, followed by a Razorpay checkout with a per-member status checklist.
- Admin Dashboard: tabular management views with inline edit and override actions.

4.2 Hardware Interfaces

None — any network-connected device with a browser.

4.3 Software Interfaces

Interface	Purpose
FretBox	Source of student and hostel master data; imported, never hand-entered
PostgreSQL	Primary data store — users, groups, hostel hierarchy, bookings, payments
Redis	Distributed locking for the allocation critical section; occupancy caching
Razorpay API	Payment order creation, checkout, and webhook-based verification

4.4 Communications Interfaces

- REST over HTTPS (JSON) between the Next.js frontend and the FastAPI backend, plus a signature-verified Razorpay webhook for asynchronous payment confirmation and a scheduled FretBox data sync.

5. Non-Functional Requirements

5.1 Performance

An allocation decision must resolve in under 500 ms under normal load, with at least 200 concurrent booking attempts handled without double-allocation.

5.2 Security

- Passwords hashed with bcrypt; JWT tokens with expiry and a refresh flow; role-based access control on all admin endpoints.
- Razorpay payment signatures verified server-side before a payment is marked complete.

5.3 Reliability & Availability

Reservation locks auto-expire (TTL, configurable, default 15 minutes) if payment is not completed, releasing the slot back to the pool and preventing deadlock or starvation.

5.4 Maintainability

A layered backend (routers $\rightarrow$ services $\rightarrow$ repositories) isolates the allocation algorithm as an independently testable module; hostel data is refreshed from FretBox by configuration, not code change.

5.5 Portability

Fully containerized; runs identically on any Docker-enabled host via `docker compose up`.

5.6 Usability

Responsive design for mobile and desktop, following familiar booking-platform conventions to minimise onboarding friction.

5.7 Business Rules

- A booking is Confirmed only when every group member's payment status is paid, and no group may exceed 4 members.
- Only an Admin or Warden may reassign a confirmed booking or edit master hostel data.

6. System Architecture (High-Level)

The Next.js frontend communicates over REST/JSON (HTTPS) with the FastAPI backend, which contains the routers, services and allocation engine. The backend persists to PostgreSQL, uses Redis for locks and caching, imports master data from FretBox, and calls Razorpay for payments. All services are orchestrated by Docker Compose.

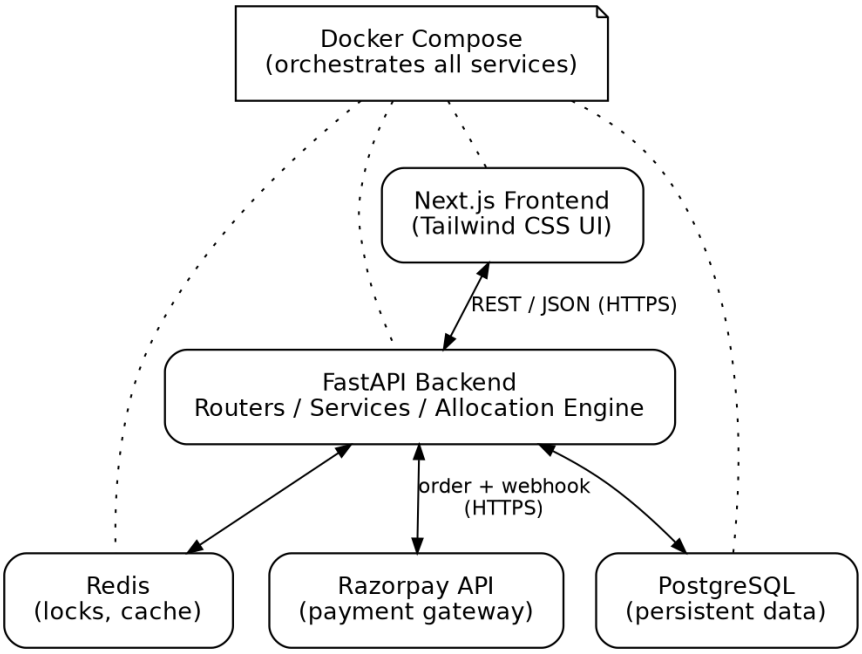


Figure 6.1: High-level system architecture and service communication.

7. Data Model (Entity Overview)

Entity	Key Attributes
User	id, name, roll_no, email, password_hash, role, gender, year, branch (synced from FretBox)
Group / GroupMember	group: id, code, leader_id, size, status | member: group_id, user_id, payment_status
Hostel / Floor / Cluster / Room	hostel: id, name, gender, year | floor: hostel_id, floor_number | cluster: floor_id, cluster_number | room: cluster_id, room_number, capacity=2, occupied_count
Preference	id, group_id, cluster_id, rank
Booking / Payment	booking: id, group_id, cluster_id, room_ids[], status, created_at (FCFS) | payment: id, user_id, razorpay_order_id, amount, status

8. Allocation Algorithm — Detailed Logic

Inputs: a group (size and ranked cluster preferences) and the current occupancy snapshot. For each preferred cluster, in rank order:

1. Lock the cluster ID to prevent concurrent changes.
2. Check occupancy against group size per FR-4.1 to FR-4.3: 4-member groups need both rooms empty; 3-member groups need the leader's room empty and the other empty or holding one occupant; 2-member groups need only the leader's room empty; singles need exactly one free seat.
3. If it fits, create a Reserved (Awaiting Payment) booking, timestamp it for FCFS, and release the lock; if not, release the lock and try the next preference.
4. If no preference fits, search all clusters for a free slot, or flag the group for Admin allocation.
5. Once every member pays within the TTL the booking is Confirmed; otherwise it is released back to the pool.

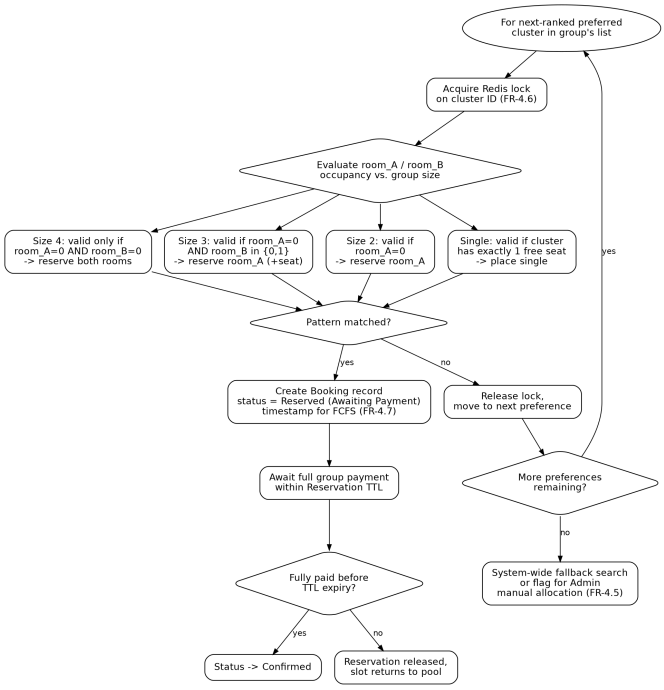


Figure 8.1: Allocation algorithm decision flow, evaluated per ranked cluster preference.

Edge cases: competing 3-member groups are resolved by FCFS booking-ready timestamp; a single applicant with no matching seat in any preferred cluster falls back to a system-wide search (FR-4.4).

9. Appendix

9.1 Tech Stack Summary

Layer	Technology
Frontend	Next.js (App Router), Tailwind CSS
Backend	FastAPI (Python), Pydantic, JWT (OAuth2 password flow)
Data	PostgreSQL (persistence), Redis (locking and cache), FretBox (master data source)
Integration / Deployment	Razorpay payments; Docker and Docker Compose